

# **Municipalidad Valle del Sol – Plataforma inteligente para la gestión y prevención de incendios.**

**Nombre: Darko Rodriguez.**  
**Desarrollo Full Stack III 003D**  
**Profesor: Claudio Esteban Rojas.**

# Índice.

|                                                   |   |
|---------------------------------------------------|---|
| Contextualización y Problemática .....            | 3 |
| Requerimientos del Sistema .....                  | 3 |
| Propuesta de Arquitectura .....                   | 4 |
| Diagrama de Contexto (C1).....                    | 4 |
| Diagrama de Contenedores (C2).....                | 5 |
| Diagrama de Componentes (C3):.....                | 6 |
| Patrones de Arquitectura y Diseño .....           | 6 |
| Patrones de Arquitectura .....                    | 7 |
| Patrones de Diseño .....                          | 7 |
| Modelo de Datos y Estrategia de Persistencia..... | 7 |
| Decisiones Críticas y Justificación Técnica ..... | 8 |
| Riesgos Tecnológicos y Mitigaciones.....          | 8 |
| Endpoints Nucleares del Sistema.....              | 9 |
| Conclusión .....                                  | 9 |
| Repositorio.....                                  | 9 |

## Contextualización y Problemática

En los últimos años, los incendios forestales y urbanos han representado una amenaza creciente para la seguridad de la población, la infraestructura y el entorno natural a nivel país, causando catástrofes de alto impacto. La Municipalidad de Valle del Sol busca modernizar sus capacidades de respuesta y mitigación de siniestros mediante el desarrollo de una plataforma tecnológica que permita mejorar la detección, el monitoreo y la respuesta ante emergencias. Actualmente, la gestión de incidentes presenta importantes limitaciones, tales como el uso de canales no estructurados (llamadas telefónicas o reportes informales en redes sociales), la falta de integración entre organismos de emergencia y la inexistencia de sistemas centralizados que permitan un análisis histórico de los eventos. Frente a este escenario, la municipalidad se ha propuesto desarrollar un sistema que resuelva directamente las siguientes problemáticas:

- Falta de herramientas digitales centralizadas para el reporte ciudadano de incendios.
- Dificultad operativa para visualizar la ubicación exacta de los focos de incendio en tiempo real.
- Escasa integración de datos entre la municipalidad y los organismos de primera respuesta.
- Falta de mecanismos automatizados para alertar a la población afectada cuando la emergencia escala.
- Ausencia de un repositorio de datos histórico que permita analizar patrones o recurrencias para la prevención.

## Requerimientos del Sistema

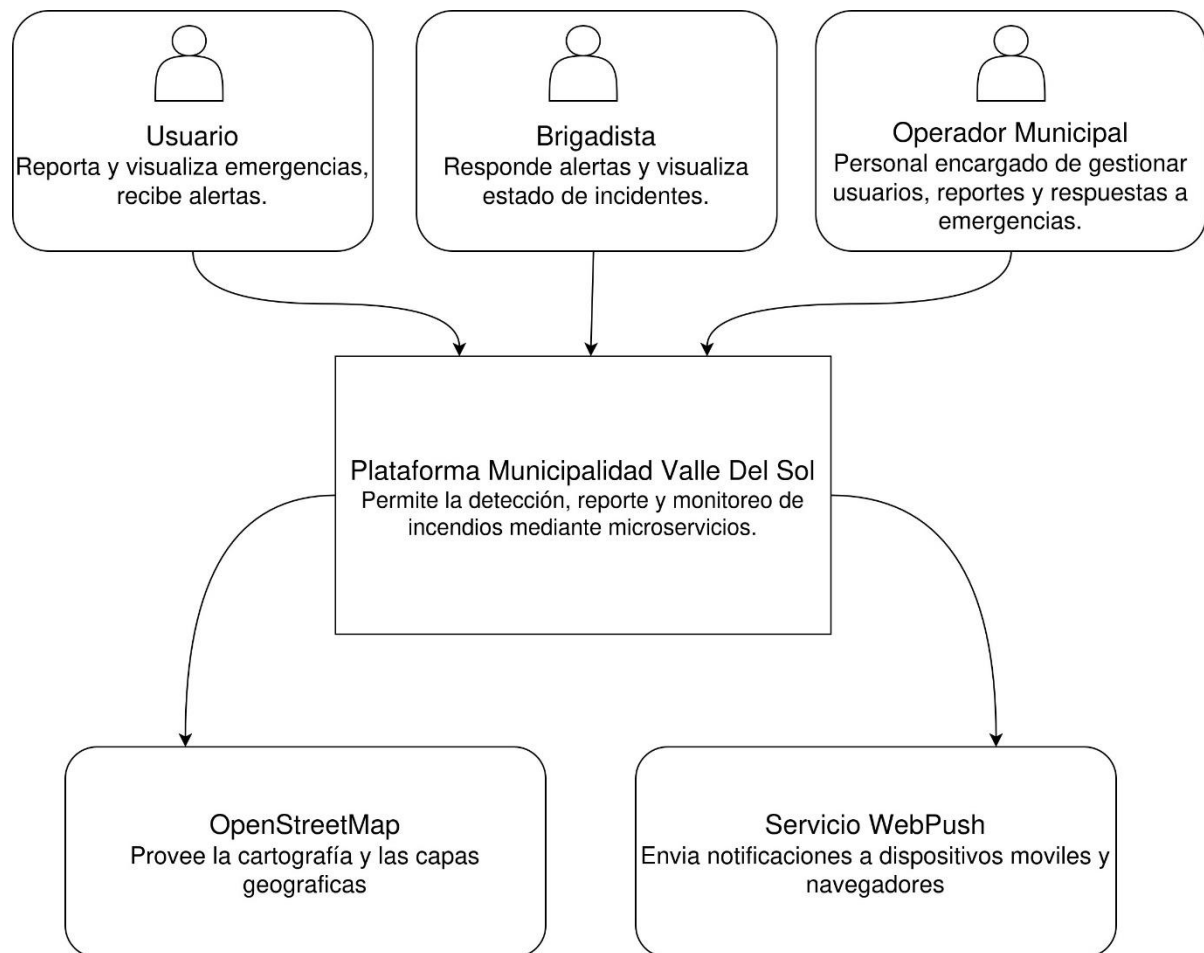
El sistema debe cumplir con tres funcionalidades principales: la detección y reporte de incendios, el mapeo cartográfico en tiempo real y la emisión de alertas a la comunidad. Estas capacidades implican la necesidad de manejar múltiples fuentes de datos, procesar información de manera inmediata y garantizar una comunicación eficiente con los usuarios.

- **Requerimientos Funcionales:** El sistema debe permitir el registro de reportes con información geográfica y multimedia adjunta, la visualización de incendios en mapas interactivos y la notificación oportuna a la población ante situaciones de riesgo. Asimismo, debe estructurar y almacenar la información histórica de los eventos para su posterior análisis estadístico.
- **Requerimientos No Funcionales:** Se destacan la necesidad de alta disponibilidad (especialmente en contextos de crisis donde el sistema no puede caer), escalabilidad elástica ante aumentos abruptos de la demanda, seguridad estricta en el manejo de datos de geolocalización y un bajo tiempo de respuesta (baja latencia) en la transmisión de datos.

La arquitectura de microservicios seleccionada permite desacoplar estas funcionalidades, escalar los componentes de manera independiente y mejorar la resiliencia global frente a fallos. A diferencia de una estructura monolítica tradicional, los microservicios facilitan la evolución continua del sistema y su integración con nuevas entidades externas o sensores IoT en el futuro.

# Propuesta de Arquitectura

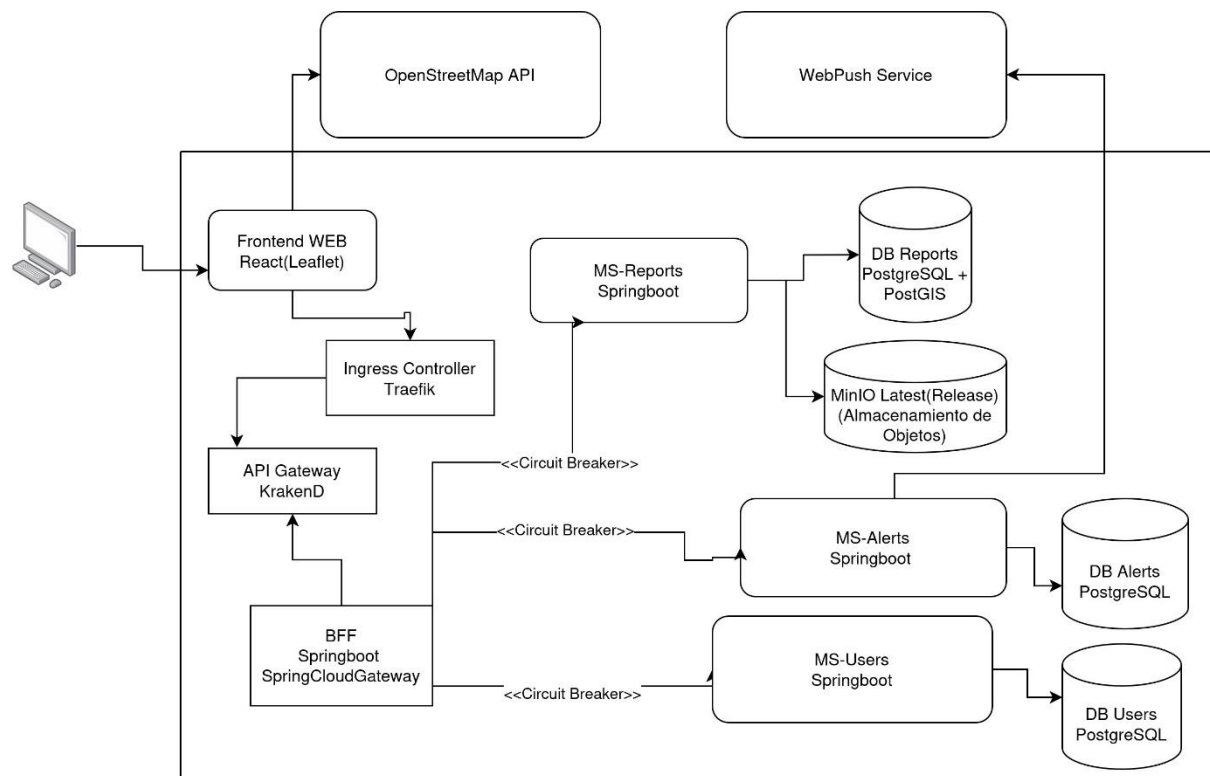
## Diagrama de Contexto (C1)



La plataforma actúa como el núcleo operativo donde convergen tres actores principales: los Ciudadanos, que generan reportes de incidentes con geolocalización; los Brigadistas, que coordinan la mitigación y actualizan los estados; y los Administradores, que gestionan usuarios y auditan el sistema. El ecosistema interactúa con dos dependencias externas críticas: OpenStreetMap (proveedor de las capas base cartográficas) y los servicios de Notificaciones Push nativos de los navegadores comerciales para alertas masivas.

En cuanto a la persistencia y el frontend, el diseño implementa el patrón de base de datos por servicio, utilizando PostgreSQL con la extensión PostGIS para transformar el almacenamiento relacional en un sistema de información geográfica capaz de gestionar datos espaciales complejos. Los archivos multimedia se centralizan en MinIO, mientras que la interfaz de usuario se concentra en un frontend único.

## Diagrama de Contenedores (C2)

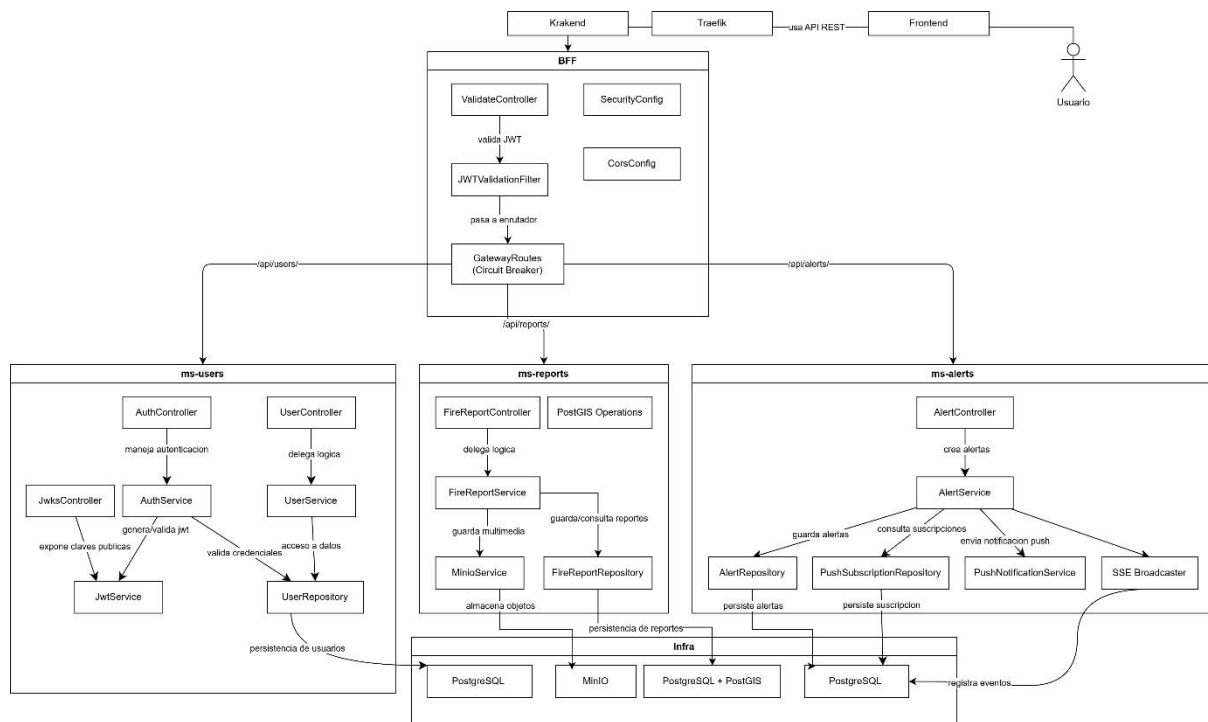


El sistema divide sus responsabilidades en componentes autónomos y completamente desacoplados:

- Frontend: Aplicación de página única (SPA) desarrollada en React, TypeScript y Vite. Gestiona la interfaz de usuario, la captura de coordenadas del dispositivo y la visualización de mapas interactivos mediante Leaflet.
- Ingress Controller (Traefik): Actúa como el punto de acceso perimetral al clúster de Kubernetes, asumiendo la responsabilidad exclusiva de gestionar el enrutamiento de red de entrada, el balanceo de carga entre nodos y la terminación automática de los certificados de seguridad TLS.
- API Gateway Perimetral (KrakenD): Proxy inverso de alto rendimiento escrito en Go. Actúa como frontera de la API gestionando la seguridad perimetral, la validación genérica y el control de tasa de peticiones (rate limiting).
- Backend for Frontend (BFF): Componente desarrollado en Spring Cloud que actúa como mediador exclusivo del frontend. Se encarga de la agregación de respuestas de múltiples servicios, traducción de protocolos y orquestación de la resiliencia fina.
- Microservicios de la aplicación (Spring Boot):
  - ms-users: Gestión de identidades, control de acceso basado en roles (RBAC) y emisión de credenciales criptográficas.
  - ms-reports: Ingesta de incidentes, almacenamiento de coordenadas espaciales y gestión de archivos.
  - ms-alerts: Creación, difusión masiva y persistencia de alarmas comunitarias.

- Infraestructura de Persistencia: Instancias independientes de PostgreSQL (optimizadas con PostGIS) y almacenamiento de objetos MinIO compatible con las APIs de AWS S3 para archivos multimedia. Todo orquestado localmente.

## Diagrama de Componentes (C3):



Estos servicios ejemplifican el estándar de desarrollo interno de la plataforma mediante una separación estricta de responsabilidades en cuatro capas operativas:

Capa de Controladores: Expone los endpoints REST convencionales y maneja la recepción de archivos multimedia mediante peticiones multipartes.

Capa de Servicios: Encapsula las reglas de negocio, incluyendo la validación de datos, la asignación de estados del incendio, la autorización y la orquestación de notificaciones internas hacia otros servicios.

Capa de Repositorios: Abstrae el acceso físico a los datos mediante Spring Data JPA, ejecutando consultas espaciales nativas directamente en el motor de base de datos.

Adaptadores de Integración: Componentes aislados que gestionan la comunicación con tecnologías externas (cliente nativo de MinIO, cliente WebPush y abstracciones HTTP mediante RestTemplate para invocar internamente a ms-alerts).

## Patrones de Arquitectura y Diseño

Para garantizar un diseño robusto, tolerante a fallos y mantenible en el tiempo, se han implementado los siguientes patrones:

## Patrones de Arquitectura

- Microservices & Database per Service: Aísla por completo el almacenamiento y la lógica de cada dominio, impidiendo que picos de tráfico de ciudadanos reportando una emergencia bloqueen las bases de datos de usuarios o alertas.
- Seguridad Centralizada (JWT & RBAC): Control de acceso basado en tokens seguros que inyectan el Rol de usuario (USUARIO, BRIGADISTA, ADMINISTRADOR) permitiendo restringir rutas administrativas y de personal calificado.

## Patrones de Diseño

- Gateway + BFF Pattern: Divide el control de acceso en dos niveles. KrakenD filtra y protege la red a nivel perimetral de forma ultra veloz, mientras que el BFF de Spring Cloud adapta los datos específicamente a lo que el mapa de React necesita mostrar en pantalla.
- Repository & Data Transfer Objects (DTO): Desacopla el modelo físico de la base de datos de la lógica de red. Las entidades internas nunca se exponen al exterior, utilizándose DTOs gestionados por fábricas de instanciación para el intercambio seguro de datos.
- Circuit Breaker (Disyuntor): Implementado mediante Resilience4j en la capa del BFF. Protege al sistema de fallos en cascada; si el servicio de alertas se degrada, el disyuntor se abre y evita que el resto de la plataforma colapse.
- Server-Sent Events (SSE) & Adapter: SSE proporciona un canal de comunicación unidireccional y persistente ideal para actualizar los mapas de las brigadas en tiempo real sin el consumo excesivo de recursos de WebSockets, mientras que el patrón Adapter aísla las APIs de los proveedores de almacenamiento (MinIO/S3).

## Modelo de Datos y Estrategia de Persistencia

Cada microservicio gestiona su propia base de datos PostgreSQL, garantizando la independencia y el aislamiento de los esquemas:

- ms\_usuarios: Almacena perfiles, números de contacto y credenciales protegidas con funciones hash de un solo sentido (BCrypt), optimizada con índices B-tree en campos de búsqueda frecuente como el nombre de usuario y correo electrónico.
- ms\_mapeo (con extensión PostGIS): Diseñada para la persistencia de la entidad FireReport. Configura el atributo location bajo el tipo geométrico GEOMETRY(POINT, 4326). Cuenta con índices espaciales indexados por árboles GIST, optimizando cálculos geométricos en tiempo real como búsquedas por radio de afectación o contención perimetral comunal.
- ms\_alerts: Almacena el historial cronológico de alertas emitidas y la entidad PushSubscription (endpoints, llaves criptográficas p256dh y firmas de autenticación de los navegadores).

Para mantener el rendimiento de la base de datos relacional, las imágenes y videos de los incendios no se guardan como bloques binarios (BLOBs). Se almacenan de forma aislada en un bucket de MinIO y la base de datos registra únicamente una URL de acceso temporal con tiempo de expiración controlado.

## **Decisiones Críticas y Justificación Técnica**

La aplicación contara con un despliegue On-Premise con Kubernetes y MinIO, y se rechazó el uso de nubes públicas comerciales (AWS/GCP) para garantizar la soberanía absoluta de los datos de la comuna, asegurar el cumplimiento de normativas gubernamentales sobre privacidad y eliminar costos variables por transferencia de datos. Al alojar la infraestructura en servidores municipales, se asegura que la información sensible y multimedia no salga del control institucional.

Se usará PostgreSQL con PostGIS para Geolocalización, esto seleccionado por encima de soluciones NoSQL debido a su madurez estructural, consistencia transaccional (ACID), precisión matemática en cálculos topológicos y soporte nativo del estándar GeoJSON requerido por el frontend.

Para las notificaciones push usaremos una estrategia dual de notificaciones (SSE + WebPush), donde se implementa SSE para mantener actualizados los dashboards de las brigadas con conexiones persistentes de bajo overhead, combinándolo con notificaciones WebPush (firmadas con llaves criptográficas VAPID) para alcanzar los dispositivos móviles de los ciudadanos de forma nativa, incluso si el navegador está cerrado.

La seguridad sera abordada mediante JWT RS256 + RBAC, y la autenticación utiliza firmas asimétricas. El microservicio ms-users expone un endpoint JWKS público, de este modo, el API Gateway y los microservicios validan la autenticidad de los tokens de forma descentralizada sin consultar constantemente al servicio de usuarios. El control de acceso basado en roles (RBAC) limita las acciones críticas exclusivamente a los perfiles BRIGADIST y ADMINISTRATOR.

## **Riesgos Tecnológicos y Mitigaciones**

Existe la posibilidad de saturación por picos masivos de tráfico en situaciones de emergencia, pero este riesgo es mitigado mediante la configuración de Escalado Horizontal de Pods (HPA) en Kubernetes, el uso de mecanismos de rate limiting aplicados en la capa perimetral de KrakenD y la configuración de pools dinámicos y optimizados de conexiones en las bases de datos.

La exposición de datos geográficos o personales sensibles es mitigada mediante la obligatoriedad de cifrado de datos en tránsito a través de HTTPS, controles de acceso RBAC estrictos a nivel de endpoint y procesos automáticos de anonimización de coordenadas al extraer información para análisis históricos.



## Endpoints Nucleares del Sistema

POST /api/users/auth/login — Autentica credenciales y emite tokens JWT RS256.

GET /.well-known/jwks.json — Publica las claves públicas para la validación descentralizada de tokens.

POST /api/reports/upload — Ingesta de reportes geolocalizados con adjuntos multimedia (Multipart/form-data).

PUT /api/reports/{id}/status — Actualiza el estado operativo del incidente. (Requiere rol BRIGADIST).

POST /api/alerts — Creación y emisión de alertas de emergencia masivas a la comunidad. (Requiere rol BRIGADIST).

GET /api/alerts/stream — Canal SSE para la actualización cartográfica en tiempo real de los tableros de control.

## Conclusión

La arquitectura propuesta para la Plataforma Inteligente de Valle del Sol resuelve de manera robusta el balance entre procesamiento en tiempo real, tolerancia a fallos y soberanía tecnológica. El diseño modular en microservicios, blindado perimetralmente por una estrategia de tres capas (Traefik → KrakenD → BFF), asegura que la municipalidad pueda absorber incrementos críticos de tráfico durante catástrofes sin comprometer la integridad del sistema.

## Repositorio.

Enlace del repositorio: <https://github.com/DarkoRodriguez/plataforma-incendios-valle-del-sol>